# From Sequences to Images: A Systematic Study of TimesNet Variants for Weather Forecasting

L. Fardin[1], R. Pesce[2], M. Saccon[3]

***Abstract*—Time series forecasting is a task in which purely sequential models often struggle to exploit multi-periodic structure directly. TimesNet addresses this by transforming one-dimensional time series into two-dimensional representations via FFT-based period discovery, allowing standard 2D convolutional backbones to capture both intra-period and inter-period variation simultaneously. From this perspective, the forecasting problem effectively becomes a computer vision problem. In this work, we present a from-scratch PyTorch reimplementation of TimesNet and systematically compare four architectural variants. We further compare against PatchTST, an attention-based architecture with no hardcoded periodicity assumption, as an architecturally distinct baseline. By comparing these architectures, we discuss how they are structured and why they perform differently. Moreover, we study how correlation between different features affects the network's performance, as well as the impact of input length and prediction horizon on accuracy. We compare the accuracy of the network trained on the full 21 features of the dataset against a version trained after dimensionality reduction via PCA, retaining 9 components that capture approximately 98% of the variance, showing that preventing the network from learning linear correlations between features can improve its performance. Our results show that, while Net1 performs best in the single-channel setting, Net2 consistently achieves the lowest error in the multi-channel setting, arguably the more relevant scenario given the correlation between features discussed above.**

***Index Terms*—TimesNet, Time series, CNN, Transformers**

## I. Introduction

Time series forecasting is a problem relevant to a wide range of applications, from traffic management to climate science. A key difficulty in long-horizon time series forecasting is capturing *multi-period temporal structure*: relevant patterns can repeat at multiple, possibly incommensurate frequencies. In this work, we investigate TimesNet [1], a recently proposed architecture that addresses this limitation by transforming one-dimensional time series into two-dimensional representations through FFT-based period discovery, enabling standard 2D vision backbones to simultaneously capture both within-period (intraperiod) and across-period (interperiod) variation. Our contribution is a systematic empirical study of four architectural variants of TimesNet.

The main contributions of this paper are:

- A from-scratch PyTorch reimplementation of TimesNet, including the FFT-based period discovery, 2D reshaping, and Inception-based feature extraction pipeline, validated on a long multi-year weather benchmark.
- A systematic comparison of four architectural variants: (*i*) the base design, where the projection is applied first and TimesBlocks refine the extended sequence; (*ii*) a design with temporal feature extraction preceding the predictive projection; (*iii*) a weight-sharing variant in which a single TimesBlock and a single linear layer are reused iteratively, reducing parameter count; and (*iv*) a cross-channel variant that processes each input channel through its own dedicated TimesBlock pipeline before merging into a joint representation.
- A comparison between two different data extraction approaches in the multichannel version of TimesNet.
- A comparison against PatchTST [2] as an architecturally distinct baseline, providing context for where the TimesNet family of designs stands relative to patch-based transformer approaches on the same task and dataset.

The remainder of this paper is organized as follows. Section II reviews related work on time series forecasting architectures, from recurrent and Transformer-based models to the periodicity-based approach introduced by TimesNet. Section III describes the four architectural variants of TimesNet studied in this work, together with the PatchTST [2] baseline. Section IV presents the dataset used for our experiments and the preprocessing steps applied to it, including the PCA-based dimensionality reduction. Section V details the learning framework shared across all variants, covering period discovery, the InceptionBlock, the TimesBlock, and the optimization procedure. Section VI reports the experimental results, first on the single-channel and then on the multi-channel setting, and Section VII concludes the paper with a discussion of our findings and directions for future work.

## II. Related Work

Time series forecasting has been extensively investigated using a wide range of deep learning architectures. Early approaches were mainly based on recurrent neural networks (RNNs), including Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models, which demonstrated the ability to capture temporal dependencies in sequential data [3], [4]. More recently, Transformer-based architectures have achieved state-of-the-art performance by exploiting self-attention mechanisms to model long-range dependencies more effectively [5]–[7].

A different perspective was introduced by TimesNet [1], which reformulates one-dimensional time series into two-

[1]Department of Physics, University of Padova,
email: leonardo.fardin.1@studenti.unipd.it
[2]Department of Information Engineering, University of Padova,
email: riccardo.pesce.4@studenti.unipd.it
[3]Department of Physics, University of Padova,
email: mattia.saccon@studenti.unipd.it

dimensional temporal representations. Rather than treating forecasting as a purely sequential modeling problem, TimesNet exploits the observation that temporal signals often exhibit multiple periodic patterns. By reshaping the input sequence according to its dominant periods, the resulting two-dimensional representation enables the use of convolutional operations originally developed for computer vision tasks. This approach allows local temporal variations and inter-period dependencies to be learned simultaneously, leading to significant improvements across several time series analysis tasks.

The convolutional backbone employed by TimesNet is inspired by the Inception architecture [8], whose multi-scale convolutional design enables the extraction of features at different receptive fields. Inception blocks have proven highly effective in computer vision because they simultaneously capture fine- and coarse-grained spatial information while maintaining computational efficiency. Their adoption within TimesNet demonstrates that multi-scale convolutions can also provide a powerful inductive bias for temporal data represented in two dimensions.

Despite these advances, the original TimesNet architecture leaves room for further improvements. In particular, the feature extraction pipeline relies on a fixed Inception block design that does not explicitly investigate the impact of different activation functions and architectural hyperparameters on forecasting performance. The present work extends this line of research by systematically exploring alternative architectural configurations while preserving the core idea of TimesNet, with the objective of improving feature representation and predictive accuracy.

## III. Processing Pipeline

In this work we faced the time series foreasting problem using two different families of architectures: TimesNet variants and PatchTST. The first approach, TimesNet, is built around the concept that periodicity is the dominant structural signal worth exploiting, and the model is architecturally designed around detecting and leveraging it directly. An alternative approach, PatchTST, makes a comparatively weaker assumption: rather than hardcoding what kind of structure matters, it relies on a general purpose attention mechanism to discover whatever relationships in the data turn out to be useful.

Hyperparameters (learning rate, embedding dimension, number of retained periods, kernel sizes, number of timesblock and so on) are tuned via Optuna [9] before the final model is trained, studied and tested on the selected configuration.

### A. TimesNet

At high level, TimesNet works by finding the most prominent cyclic patterns in the sequence and using that information to produce a representation of the data that might expose predictive features relative to each pattern. More concretely, it finds dominant frequencies in the signal and reshapes to a 2D representation splitting it at specific intervals dictated by each found frequency, and then stacking each chunck, exposing inter-periodic patterns.

- **Net1bis:** The base version first generates a rough prediction through a linear layer, which is then refined by a stack of TimesBlocks. Since past and predicted timesteps are processed together, the model can exploit periodic patterns in the input sequence to improve the forecast. The internal operations of the TimesBlock are described later.
- **Net1:** This variant simply swaps the order of the blocks, placing the predictive linear layer after the TimesBlock pipeline. Consequently, feature extraction is performed only on the input sequence, while the processed representation is finally projected into the future prediction.
- **Net2:** Instead of stacking different TimesBlocks, this architecture repeatedly applies the same TimesBlock and the same linear layer, learning a single transformation that is iteratively refined.
- **Net3:** This variant processes each channel independently through its own TimesBlock pipeline before combining them into a shared pipeline, delaying cross-channel interactions until channel-specific features have been extracted.

### B. PatchTST

The last network we tried (**Net4**) is an implementation of PatchTST [2], a transformer network that applyes self attention to small chuncks of the input sequence (patches) to find predictive patterns, with no hardcoded knowledge of the kind of patterns it is supposed to find, leaving pattern discovery entirely to the attention mechanism, in contrast to TimesNet's explicit periodicity assumption.

## IV. Signals and Features

The data used in this work was downloaded from the Max Planck Institute for Biogeochemistry's Jena Beutenberg weather station (https://www.bgc-jena.mpg.de/wetter/weather_data.html). The dataset consists of a 21-feature time series recorded starting in 2003 and continuing to the present. We used data collected from 2008 to 2025: this period was chosen for reproducibility purposes. Setting 2025 as the final year ensures that execution results do not depend on the state of the online-available data or on the time at which execution begins; 2008 was chosen as the starting point because it is the first year for which all 21 features are available.

Excluding particular case studies, we used 96-timestep-long time series as input to predict the following 96 timesteps. The total time window for each sample is therefore a 192-timestep-long sequence with 21 features. Since the distance between one time step and the following is 10 minutes, the prediction lasts 16 hours.

To construct the dataset, the $2008\hat{a}\backslash 2025$ time series was split into sequential 192-timestep windows. A sliding-window (slicing) procedure was also applied to enrich the dataset: the time window is shifted forward by a fixed number of timesteps

to generate additional, partially overlapping samples. To avoid loading the entire dataset into memory at once, slicing was performed in four stages, shifting the window forward by 48, 96, and 144 timesteps, respectively. No missing values were found in the selected period.

After generating the samples, the dataset was shuffled and split into training and validation sets using an $80\% - 20\%$ ratio (63098 training samples, 15775 validation samples). We used the 2026 data as the test set.

Since the number of network parameters grows with the number of input features, we investigated whether reducing the degrees of freedom significantly affected the network's loss. We applied Principal Component Analysis (PCA) and found that approximately $98\%$ of the variance is captured by just 9 components. We therefore trained the network on the PCA-projected-normalized-scaled samples and applied the inverse transform to the output before comparing it with the original data. This approach is compared against the full 21-feature setting in Fig. 4.

*a) Applying PCA to the multichannel network:* Prior to training, the 21-dimensional feature space is reduced via Principal Component Analysis (PCA) to retain only the most informative components. The PCA transformation matrix is computed exclusively on the training dataset, following standardization of the data (zero mean, unit variance) via a StandardScaler fitted on the same training split. Both the scaler and the PCA matrix are then applied to transform the training data into the reduced representation, on which the network is subsequently trained. The fitted scaler and PCA matrix are stored alongside the model weights and reused, never recomputed, at evaluation time, ensuring a leakage-free mapping between feature spaces.

## V. Learning Framework

All four architectural variants studied in this work share the same two core building blocks: the *TimesBlock*, responsible for discovering dominant periodicities and reshaping the signal into a two-dimensional representation, and the *InceptionBlock*, responsible for extracting temporal features from that representation through parallel multi-scale convolutions. Since these two components are reused, unmodified, across every variant, we describe them here in detail; how each variant assembles them into a full forecasting model is described in Section III.

### A. Period discovery via FFT

Given an input tensor $\boldsymbol{X} \in \mathbb{R}^{T \times C}$, where $T$ is the sequence length and $C$ is the number of feature channels at that stage of the network (i.e. the embedding dimension $d_{model}$, not the number of raw input variates), the model first identifies the $k$ dominant frequencies present in the signal via the Fast Fourier Transform. For each channel, the amplitude spectrum is computed and averaged across channels, and the top-$k$ frequencies by amplitude are retained:

$$\{f_1, \ldots, f_k\}, \{a_1, \ldots, a_k\} = \underset{f}{\operatorname{argmax}^k} \left( |\mathrm{FFT}(\boldsymbol{X})| \right), \quad (1)$$

where $a_i$ denotes the (unnormalized) amplitude associated with frequency $f_i$. Each retained frequency $f_i$ is converted into a corresponding period length

$$p_i = \lfloor |\mathrm{round}(f_i \cdot T)| \rceil, \qquad p_i \geq 1, \quad (2)$$

and the amplitudes are normalized into per-period importance weights via a softmax,

$$w_i = \frac{\exp(a_i)}{\sum_{j=1}^{k} \exp(a_j)}, \qquad i = 1, \ldots, k. \quad (3)$$

### B. 1D-to-2D reshaping

For each of the $k$ candidate periods $p_i$, the sequence is reshaped from its native one-dimensional form into a two-dimensional tensor whose two axes separately capture *intraperiod* variation (variation within a single period, e.g. the daily temperature cycle) and *interperiod* variation (variation across repetitions of that period, e.g. how successive days differ). Concretely, $\boldsymbol{X}$ is zero-padded along the temporal axis so that $T$ becomes an integer multiple of $p_i$, then reshaped and permuted into

$$\boldsymbol{X}_{2D}^{(i)} \in \mathbb{R}^{C \times n_i \times p_i}, \qquad n_i = \lceil T/p_i \rceil, \quad (4)$$

where $n_i$ is the number of periods contained in the (padded) sequence. When a candidate period is longer than half the sequence length (i.e. fewer than two full periods fit in the input), the reshape step is skipped and the sequence is instead treated as a single column ($n_i = T$, $p_i = 1$), degenerating gracefully to the original 1D ordering. This produces a set of $k$ two-dimensional tensors $\{\boldsymbol{X}_{2D}^{(1)}, \ldots, \boldsymbol{X}_{2D}^{(k)}\}$, one per candidate period, each ready to be processed by 2D convolutional kernels.

### C. InceptionBlock: multi-scale feature extraction

Each 2D tensor $\boldsymbol{X}_{2D}^{(i)}$ is independently processed by the same InceptionBlock, shared across all $k$ periods. The InceptionBlock applies a set of parallel 2D convolutions with kernel sizes $\mathcal{K} = \{1, 3, 5, 7\}$ (kept identical for all input/output channels via `padding='same'`), and averages their outputs:

$$\mathrm{Inception}(\boldsymbol{X}_{2D}^{(i)}) = \frac{1}{|\mathcal{K}|} \sum_{\kappa \in \mathcal{K}} \left( \mathrm{Conv2d}_{\kappa} \left( \boldsymbol{X}_{2D}^{(i)} \right) \right), \quad (5)$$

where $\mathrm{Conv2d}_{\kappa}$ denotes a 2D convolution with a $\kappa \times \kappa$ kernel. Using multiple kernel sizes in parallel, rather than a single fixed size, lets the block capture both short- and long-range dependencies within and across periods simultaneously, at the cost of $|\mathcal{K}|$ convolutional branches instead of one.

### D. TimesBlock: aggregation and residual connection

The TimesBlock ties together period discovery, reshaping, and feature extraction into a single residual module. Each of the $k$ processed 2D tensors is flattened back into a 1D sequence and truncated to the original length $T$ (discarding the padding introduced in Section V-B):

$$\hat{\boldsymbol{X}}_{1D}^{(i)} = \mathrm{Trunc}_T \left( \mathrm{Reshape} \left( \mathrm{Inception} \left( \boldsymbol{X}_{2D}^{(i)} \right) \right) \right) \in \mathbb{R}^{T \times C}. \quad (6)$$

The $k$ resulting sequences are then combined into a single output via a weighted sum, using the softmax weights from Eq. (3) as the mixing coefficients, and a residual connection is added around the whole block:

$$\text{TimesBlock}(\boldsymbol{X}) = \boldsymbol{X} + \sum_{i=1}^{k} w_i \cdot \hat{\boldsymbol{X}}_{1D}^{(i)}. \tag{7}$$

The residual connection in Eq. (7) is what allows TimesBlocks to be stacked or reused repeatedly (as in the weight-sharing variant, Section III) without destabilizing training, since each block only needs to learn a *correction* to its input rather than reconstructing the signal from scratch. This mirrors the residual design of the original TimesNet architecture [1], on which our InceptionBlock and TimesBlock implementations are closely based.

### E. Optimization

All variants are trained to minimize the mean squared error between predicted and ground-truth values,

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \|\boldsymbol{y}_n - \hat{\boldsymbol{y}}_n\|_2^2, \tag{8}$$

with $N$ the batch size, $\boldsymbol{y}_n$ the true future sequence, and $\hat{\boldsymbol{y}}_n$ the model's prediction. We use Adam [10] with a fixed weight decay of $\lambda = 10^{-4}$ across all variants. The remaining hyperparameters – learning rate, embedding dimension $d_{model}$, number of retained periods $k$, and InceptionBlock kernel set – are tuned separately for each variant with Optuna [9], using its default TPE sampler and default pruning to cut off unpromising trials early. No sampler or pruner settings were customized. As these tuned values differ per variant and do not fit within the page limit, we include them as supplementary material rather than in the report itself.

Every input window is normalized per-sample (zero mean, unit variance) before entering the network, and the model's output is de-normalized with those same statistics before the loss is computed. This keeps training stable despite the seasonal drift in the data, and is applied identically across all four variants.

## VI. Results

The experimental analysis was conducted in two stages. We first evaluated the proposed approach on the univariate forecasting task using only the temperature channel, where the optimal hyperparameters were identified through Optuna-based hyperparameter optimization. The study was then extended to the multivariate setting by considering all available input channels, with Optuna employed again to determine the best hyperparameter configuration.

### A. Single-channel

*a) Net1 VS Net1bis:* As a first experiment, we investigated whether the position of the prediction module within the network affected the forecasting performance. Specifically, we compared two architectures: one in which the prediction is generated before the feature refinement stage, and another in which the network first learns the underlying temporal patterns and subsequently produces the prediction. To ensure a fair comparison, both models were trained using five different random seeds. As shown in Fig. 1, no significant performance differences were observed between the two architectures. Therefore, in the remainder of this work, the two configurations will be considered equivalent. Whenever a single architecture is required, preference will be given to Net1, as it contains fewer trainable parameters and consequently requires less training time.

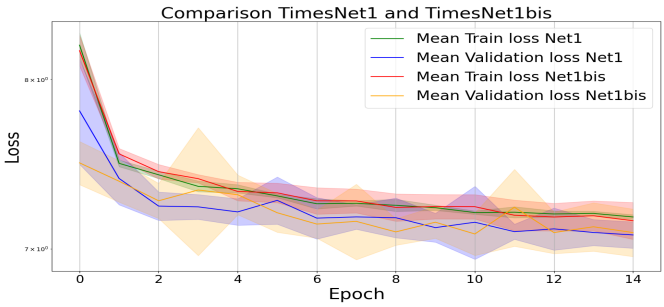



Fig. 1: TimesNet1 compare to TimesNet1Bis: the plot shows that there is no improvement by using the prediction layer after the TimesBlocks

*b) Prediction length vs validation error:* We tested how the validation error per timestep varies with the prediction length, keeping the input length fixed to 96. For each prediction length (96, 192, 384, 768) we trained 5 networks and stored the minimum validation error obtained over 10 epochs. We then computed the mean and standard deviation for each prediction length. Fig. 2 shows the results.

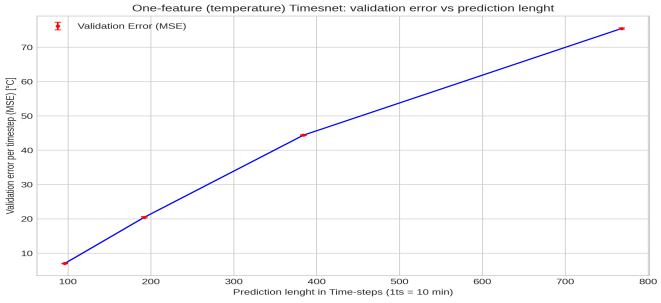



Fig. 2: Validation error per timestep vs prediction length.

As can be observed, the validation error per timestep increases as the prediction length grows.

*c) Input length vs validation error:* We tested how the validation error per timestep varies with the input length, keeping the prediction length fixed at 96. For each input length (96, 192, 384, 768) we trained 5 networks and stored the minimum validation error obtained over 10 epochs. We then computed the mean and standard deviation for each input length. Fig. 3 shows the results.

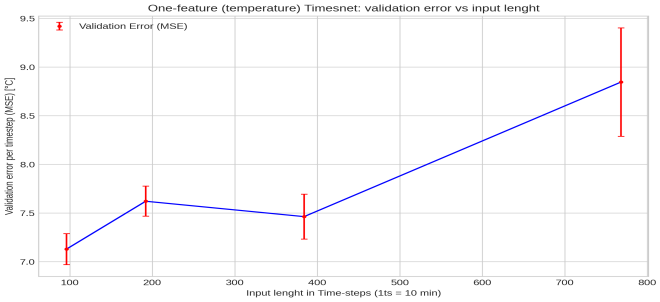



Fig. 3: Validation error per timestep vs input length.

As can be observed, the validation error appears to remain stable (at least within 3 standard deviations of the mean) as the input length increases. This suggests that a longer input length does not significantly affect the performance of the network (if anything, it may slightly worsen it, as the graph suggests).

## B. Multi-channel

*a) PCA vs full channels dataset:* We tested whether it was better to use the full 21-feature dataset or its PCA-transformed version. We found that, for this dataset, most of the information (almost 98%) is explained by 9 components. We trained 5 networks using the 21 features and another 5 using the PCA transformation for ten epochs each, then compared their validation losses. As shown in Fig. 4, the PCA-based model performs better. PCA makes learning time 1.29x faster as well.

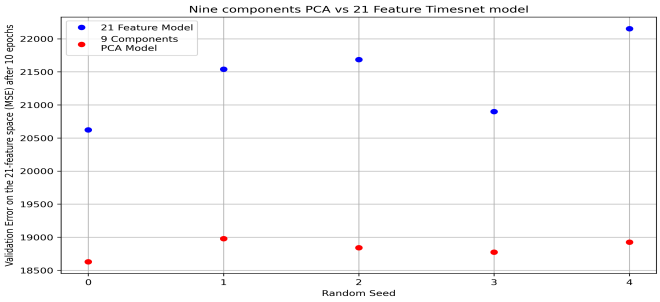



Fig. 4: Full feature model vs PCA

*b) Try a different loss function:* From Fig. 7 you can see that Net1 makes mistakes in the prediction starting from the very first prediction timestep. We asked whether reducing that error could reduce the overall error. We let the network predict one more timestep, where the very first predicted timestep should coincide with the last one it received as input. To do so, we used a weighted MSE as the loss function, where the error on the first predicted timestep is weighted 100 times more heavily than all the others. Fig. 5 shows the results.

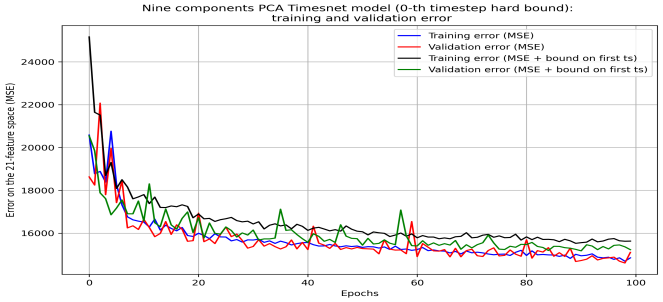



Fig. 5: Training and validation loss for the weighted and unweighted MSE loss compared

As can be observed, there is no gain in doing this.

*c) Net3:* Despite expecting improved performance by first performing an independent analysis of each input channel and subsequently combining the extracted features, thereby mitigating one of the main limitations of the original Times-Block, the averaging of the Fourier transforms across all channels, we observed poor performance from the very beginning. During the Optuna-based hyperparameter optimization, the validation loss consistently remained approximately twice as high as that achieved by the other network architectures. Given these results, we decided not to pursue further training or evaluation of this architecture.

## C. Final results

After training each network for 100 epochs using the optimal hyperparameters identified during the Optuna optimization process, we evaluated their performance on the test set. The resulting test losses for the different architectural configurations are reported in Table 1, while Fig. 7 compares the predicted and ground-truth temperature values on the test set, providing a qualitative assessment of the forecasting performance.

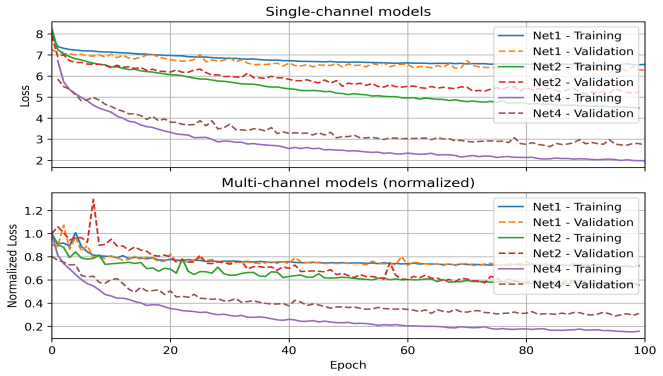



Fig. 6: Training and validation loss curves for all analyzed architectures.

| Loss | Net 1 | Net 2 | Net 4 |
|---|---|---|---|
| Single-channel | 6.233 | 8.056 | 8.887 |
| Multi-channel (only T) | 7.090 | 6.049 | 7.918 |
| Multi-channel (Total) | 14860 | 11770 | −− |

TABLE 1: Final forecasting losses for the three experimental settings

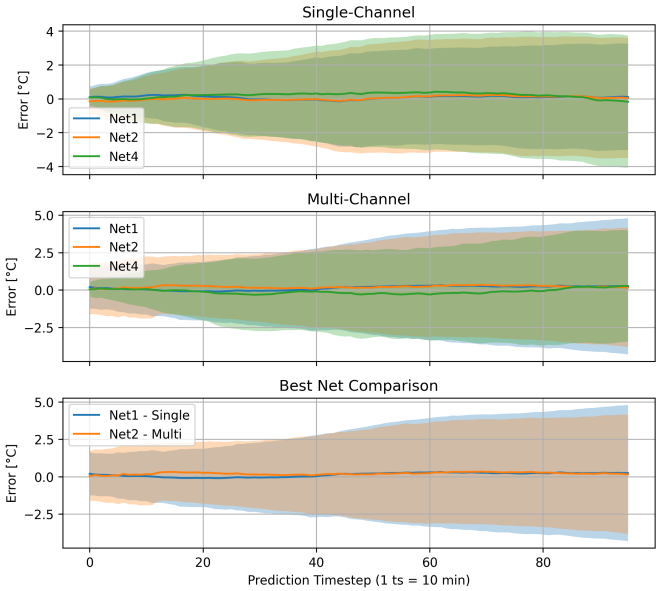



Fig. 7: Average difference between predicted and true value versus prediction timestep. Mean value line, standard deviation shaded area

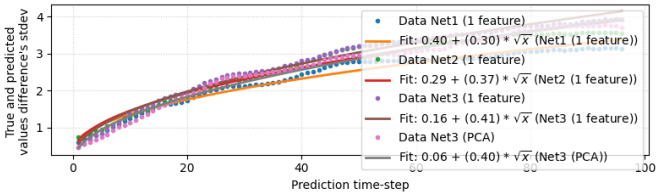



Fig. 8: Fit with square root function plus bias of the standard deviation of Fig.7

## VII. Concluding Remarks

*a) Best Network:* Among the different TimesNet architectures analyzed in this study, Network 1 performed best for the 1-feature model: its validation loss is slightly lower than the others, and its simple architecture makes it slimmer and faster to train. However, in general, Network 2 achieved the best results in both single-feature and multichannel predictions. A comment on this is provided in VII-0d.

*b) Prediction and input length:* As shown in the plots, we observe that the loss of the network grows as the prediction length increases. Since 96 was the minimum prediction length explored in the paper [1], we have stuck to it. Despite what seemed intuitive, the input length does not affect the performance of the network much (it makes it worse, as the data suggests). A possible explanation for this could be the fact that, since TimesNet architectures work with frequencies, it does not matter to have a very long input window if the main frequencies needed for a short-term forecast can be found in a few timesteps. It would be interesting for further studies to investigate this.

*c) PCA vs Full Feature Dataset:* An interesting result is the fact that PCA increases the network's performance. An explanation for this is the fact that PCA avoids the network learning linear correlations between the features, letting it learn only higher-order correlations. Further studies should investigate how the introduction of higher-order dimensionality reduction, starting from well-known physical laws, can improve the network's prediction power. However, this lies outside the scope of our work.

*d) One-Feature vs Multichannel Networks:* From Tab. 1 it is possible to observe that the Multichannel Network seems to perform slightly better than the One-Feature network. This can be explained by the fact that the correlation between channels should increase the prediction power of the network.

*e) Possible Brownian motion of the difference between true and predicted value:* As shown in Fig. 8, the standard deviation of the difference between the true and predicted values seems to follow Brownian-motion-like dynamics (zero mean and standard deviation proportional to the square root of the prediction time). Future studies should investigate whether this could be useful for enhancing the performance of the network.

## References


[1] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "Timesnet: Temporal 2d-variation modeling for general time series analysis," in *International Conference on Learning Representations*, 2023.

[2] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *International Conference on Learning Representations*, 2023.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho, B. van MerriÃnboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of EMNLP*, pp. 1724–1734, 2014.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11106–11115, 2021.

[7] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Advances in Neural Information Processing Systems*, vol. 34, pp. 22419–22430, 2021.

[8] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1–9, 2015.

[9] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.